Adams Harness interview-to-module prototype

Build a Mission Critical Safety pilot that interviews employees, produces an approved solution specification, generates a controlled source module through a sandboxed Hermes service, previews it on GitHub/Vercel, and permits an authorized admin to deploy the reviewed commit, while establishing the secure AdamsBrain foundation.

Summary

The first end-to-end proof will start in the SVP web portal and require an admin-managed department (initially **Mission Critical Safety**) before the AI asks any process questions. The interview will dynamically cover the current process, pain points, desired future state, actors/systems, notifications, completion evidence, deadlines, exceptions, and integrations; it will then produce a versioned specification for employee review and explicit approval.

Approved specifications will be sent to a separately hosted, bearer-authenticated **Hermes Agent** build service. Hermes will generate constrained TypeScript source into an isolated GitHub branch, run validation, and rely on the GitHub/Vercel integration for a preview. The portal will show the immutable commit, checks, diff/summary, and preview. An admin-only Deploy action will merge that exact reviewed commit to `main`; it will never expose Hermes terminal access or GitHub/Vercel credentials to the browser.

The generated-code contract will deliberately be narrower than arbitrary repository editing: generated modules may add a typed module package under `modules/generated/<slug>/` and update a generated registry, while shared platform code, authentication, security rules, environment/configuration, and deployment code are denied paths. The first generated acceptance module is a six-month company vehicle inspection with photo capture, VIN/license plate fields, configurable questions, seven-day reminder, supervisor escalation, and completion evidence.

Implementation Steps

- **Secure the existing application boundary before introducing agents or employee data.**
- Add a server-side Firebase authentication/authorization helper that verifies the caller's Firebase ID token with `adminAuth`, resolves the linked `users`/`teamMembers` role, and exposes `requireUser`, `requireAdmin`, and `requireSuperAdmin` guards.
- Make all new interview, AdamsBrain, build, preview, approval, and deploy endpoints server-only and guarded; the client will send its current Firebase ID token, never a claimed user/role.

- Tighten `firestore.rules` and `storage.rules` for the new collections and remove the development catch-all/open settings behavior. Platform API keys and Hermes/GitHub/Vercel secrets must move to server environment variables; the browser may receive only masked connection status.
- Correct the existing `aiEmployeeChats` versus `aiChats` rule mismatch while touching this boundary.
- Add an append-only audit collection for specification approvals, build requests, preview results, deployment attempts, and deployment outcomes.
- Treat credential rotation as a deployment prerequisite because credentials are present in repository history; do not rewrite history automatically.

- **Add Adams organization and department ownership.**
- Extend the schema with `DepartmentDoc` (`name`, `code`, `description`, owners, members, active flag) and add `departmentIds` plus `supervisorId` to team members.
- Seed Mission Critical Safety through an idempotent admin action, not a hard-coded client list, and allow admins to create/edit departments and assign owners/members.
- Add a Department admin page and sidebar/feature visibility entry.
- Make department selection the mandatory first step of every solution interview. If the user was launched from a department context, preselect it but still require confirmation.

- **Create typed interview, specification, and build lifecycle models.**
- Add Zod-validated domain types for interview sessions/messages, extracted process facts, coverage/readiness, specification versions, approvals, generated module builds, check/deploy status, and audit records.
- Use explicit state machines rather than free-form flags:
- interview: `draft -> interviewing -> ready_for_review -> completed`
- specification: `draft -> user_review -> approved | changes_requested -> superseded`
- build: `queued -> generating -> validating -> preview_ready -> deploy_approved -> deploying -> deployed | failed`
- Store every AI response and specification version with author, timestamps, department, prompt/model metadata, and a canonical hash. Approval binds to a specific spec hash; deployment binds to a specific Git commit SHA.

- **Introduce a single server-side AI/Hermes adapter.**
- Replace new direct provider calls with `lib/hermes/client.ts`, configured by `HERMES_API_URL`, `HERMES_API_KEY`, and a dedicated Adams profile/model. The Next.js server calls Hermes' OpenAI-compatible/run APIs; the browser never calls Hermes directly.
- Configure Hermes separately in Docker/VM with a distinct API key, restricted network exposure, a persistent data volume, and unattended dangerous-command mode set to deny. Use an isolated checkout/container and a least-privilege GitHub App/token that can create branches/checks and merge only through the controlled application flow.
- Keep OpenAI/Anthropic model credentials inside Hermes. Add only connection health/model status to the portal settings page and remove raw LLM secrets from Firestore-backed settings.

- Use structured JSON responses validated with Zod and retry/reject malformed output; do not parse executable instructions out of prose.

- **Build the employee process interview experience.**
- Add `/portal/solutions/new` and `/portal/solutions/[id]` with a responsive chat UI, autosaved history, progress/coverage panel, pause/resume, and explicit Finish Interview control available only when required coverage is complete.
- The interview system prompt will ask one focused question at a time and dynamically follow gaps while always covering: department; job/task/routine; trigger and frequency; current step sequence; actors and systems; copied/re-entered data; pain points/failure modes; ideal process; human approvals; notifications/handoffs; definition/evidence of done; timing/escalation; exceptions; security/compliance; and desired integrations.
- Each turn returns `{assistantMessage, extractedFacts, unresolvedQuestions, coverage, readyForSpecification}`. Merge facts deterministically and show employees what the system believes before finalization.
- Add a Solutions admin list with status, department, owner, last activity, and resume/review actions.

- **Generate, revise, and approve the solution specification.**
- Generate a structured specification containing executive summary, current-state workflow, pain points, future-state workflow, roles/RACI, data model, forms, notifications/escalations, integrations, human approval points, permissions, audit requirements, acceptance criteria, risks/assumptions, and out-of-scope items.
- Render a readable review page alongside editable structured sections. User edits create a new version; AI-assisted revision never overwrites an approved version.
- Require the interview owner to confirm factual accuracy and an authorized department owner/admin to give final delivery approval. Capture names, roles, timestamps, comments, and the approved hash.
- A later change automatically supersedes build readiness and requires reapproval before generation.

- **Create the controlled generated-module runtime and source contract.**
- Add a shared module engine that renders typed manifests for forms, records, dashboards, schedules, assignments, notification/escalation rules, completion evidence, and allowlisted integration actions.
- Define `AdamsModuleDefinition` and validate generated modules both statically and at runtime. Generated source is real compiled TypeScript, but the first prototype may only:
- create files under `modules/generated/<slug>/`;
- register the module in `modules/generated/registry.ts`;
- use approved field/action types and shared server APIs;
- add tests/fixtures for that module.
- Deny generated changes to authentication, Firestore/storage rules, package scripts/dependencies, environment files, CI/deployment configuration, shared admin code, or arbitrary network/terminal execution. Deep third-party integrations will be implemented as reviewed platform connectors and referenced by ID from generated modules.
- Add `/portal/modules/[slug]` as the stable runtime route and admin module catalog/status pages so a deployed module appears in the admin section without generating arbitrary routes.

- **Implement asynchronous Hermes -> GitHub -> Vercel generation and preview.**
- Add an admin-only Generate action that freezes the approved spec, creates a build record/idempotency key, and starts an asynchronous Hermes run with the exact module contract, allowed paths, repository/branch details, acceptance criteria, and required commands.
- Hermes creates `adams-module/<solution-id>-<spec-version>`, generates the module, runs focused tests/typecheck/lint, commits, pushes, and reports branch/SHA/results. It must stop rather than broaden paths or modify forbidden files.
- Add GitHub REST helpers and signed webhook handling to reconcile commits/check runs and capture the Vercel preview deployment URL/status. Add a cron-safe reconciliation endpoint as a fallback for lost webhooks.
- Show generation logs, changed-file list, commit SHA, checks, immutable spec hash, and Vercel preview in the portal. Failed checks or forbidden paths block approval.
- The admin Deploy button revalidates admin identity, approval, exact head SHA, allowed path diff, successful required checks, and preview success, then merges that SHA to `main` through GitHub. Record the production Vercel result and expose rollback as a new reviewed revert deployment—not a hidden destructive reset.

- **Use the vehicle inspection as the first generated acceptance module.**
- Feed a completed/approved Mission Critical Safety interview fixture through the same generation path; do not hand-create a special page that bypasses the generator.
- Expected generated definition: six-month recurrence; assigned driver/vehicle; VIN and license plate; inside/outside/required-angle photos; configurable condition questions; comments and issue severity; employee attestation; completion evidence; seven-day overdue reminder; supervisor escalation; status/dashboard filters; audit timestamps.
- Add shared photo upload support to Firebase Storage with MIME/size limits and department/module-scoped access rules.
- Add a protected scheduled processor for due/reminder/escalation actions with idempotency keys so Vercel Cron retries cannot send duplicate notices.
- For the prototype, support in-app and configured Adams email notifications; Telegram/Teams become adapters in a subsequent phase after web validation.

- **Establish the AdamsBrain foundation for uploads and approved platform data.**
- Rebrand the employee knowledge surface as AdamsBrain and replace the current mock `Ask IntellEDGE` behavior.
- Add admin-managed brains with department scope, owners, allowed audiences, source status, and freshness metadata.
- Support authenticated uploads (initially text, Markdown, PDF, and DOCX), server-side text extraction, versioning/checksums, chunk records, ingestion status, and source citations. Add approved read-only platform adapters for departments, team members, module procedures/specifications, safety calendars, and LMS/certification records; never expose records the caller cannot access.
- Retrieve scoped chunks/records, pass only those excerpts to Hermes, and require answers to include source IDs/links and an explicit insufficient-evidence response rather than fabrication.

- Add freshness/reindex controls and audit who uploaded, approved, queried, or retired knowledge. Start with uploads and platform records; Microsoft 365 ingestion is a later connector.

- **Prepare the existing requested solutions as reviewed templates, not simultaneous custom builds.**
- Add interview starter templates for Safety Request Inbox, Vehicle Inspection, Multi-site Bilingual Safety Calendar, and Training/Certification + QR verification. Each starts by asking/confirming the department and then follows the same interview/spec/approval pipeline.
- After the vehicle proof succeeds, use the generated-module contract to implement in order: Jeff/BettyJean request triage backed by AdamsBrain; safety calendar migration into portal admin plus chat commands; LMS employee certification expiry/reminders and public minimal-data QR verification.
- Keep Telegram/Teams as channel adapters to the same authenticated services so business logic is not duplicated in webhook handlers.

- **Update navigation, branding, configuration, and operational visibility.**
- Add sidebar entries for Solutions, Departments, Modules, and AdamsBrain with role/feature visibility.
- Update Adams-facing labels/assets while preserving unrelated SVP functionality during the prototype; avoid a broad white-label rewrite in this first delivery.
- Add health/status cards for Firebase Admin, Hermes, GitHub, and Vercel with masked configuration, plus build/deploy audit views.
- Extend `.env.example` with names only for Firebase Admin, Hermes, GitHub App/repository, webhook secrets, Vercel/reconciliation, and cron configuration; secrets are provisioned in Hermes/Vercel secret stores and never committed.

Files to Modify

- `lib/schema.ts` - Department, interview, specification, build, module, AdamsBrain, and audit types/collection names.
- `lib/firebase-admin.ts` - server auth/admin primitives used by guarded APIs.
- `lib/permissions.ts` - solution/module/department/brain/build/deploy permissions.
- `components/portal/portal-sidebar.tsx` - Adams Harness navigation.
- `lib/feature-visibility.ts` and related visibility UI - new feature keys.
- `contexts/user-profile-context.tsx` and `app/(portal)/portal/admin/team-members/page.tsx` - department/supervisor linkage.
- `app/(portal)/portal/settings/page.tsx` - masked Hermes/build integration health; remove client-managed AI secrets.
- `firestore.rules`, `firestore.indexes.json`, `storage.rules` - least-privilege collection, query, upload, and generated-module data access.
- `vercel.json` - guarded reconciliation and workflow schedule entries.
- `.env.example` - configuration names/documentation only.
- `package.json`, `package-lock.json` - only vetted parsing/testing dependencies required by implementation.

Files to Add

- `lib/auth/server-auth.ts` - Firebase token verification and role guards.
- `lib/adams-harness/{types,state-machines,interview,specification,module-contract,audit}.ts` - validated core domain and orchestration.
- `lib/hermes/client.ts` - server-only Hermes API/run client.
- `lib/integrations/{github,vercel}.ts` - least-privilege branch/check/preview/deploy coordination.
- `lib/modules/{registry,engine,actions,scheduler}.ts` - safe generated-module runtime.
- `modules/generated/registry.ts` and generated module packages - controlled source output.
- `lib/adams-brain/{ingestion,retrieval,sources}.ts` - scoped knowledge ingestion/retrieval with citations.
- `app/api/adams-harness/...` - authenticated interview, specification, approval, generation, status, deploy, scheduler, webhook, and AdamsBrain endpoints.
- `app/(portal)/portal/solutions/...` - employee interview and specification review flows.
- `app/(portal)/portal/admin/{departments,solutions,modules,adams-brain}/...` - admin management and deployment surfaces.
- `app/(portal)/portal/modules/[slug]/page.tsx` - shared deployed-module host.
- `app/(portal)/portal/adams-brain/page.tsx` - cited employee knowledge chat.
- Focused unit/integration tests beside core services plus vehicle fixture/contract tests (exact layout follows the selected test runner).

Verification

- [ ] Install dependencies through `npm install` / `npm add` and record lockfile changes; never hand-edit dependency versions.
- [ ] Run `npx tsc --noEmit` and resolve all new errors.
- [ ] Run `npx eslint` against changed application files (the repository's `npm run lint` currently points at a removed Next.js command and should be corrected if used).
- [ ] Add and run focused tests for Zod schemas, lifecycle transitions, interview coverage, spec hashing/version invalidation, auth/role guards, allowed-path enforcement, webhook signature verification, exact-SHA deploy gating, scheduler idempotency, and AdamsBrain access/citations.
- [ ] Use Firebase emulators or a dedicated non-production project to verify Firestore/Storage rules: employee own/department reads, admin writes, public denial, cross-department denial, and no browser access to secrets.
- [ ] Verify interview flow manually: department is required; resume works; dynamic follow-ups cover all required dimensions; premature finalize is blocked; modification creates a new spec version; dual approval captures the immutable hash.
- [ ] Run the vehicle fixture through Hermes in its sandbox and verify only the generated directory/registry changes, required checks pass, a GitHub branch/commit appears, and Vercel supplies a working preview.

- [ ] Verify Deploy rejects non-admins, stale/changed SHAs, changed specs, forbidden files, failed checks, missing preview, duplicate requests, and webhook replay; verify a valid reviewed SHA merges and production status is audited.
- [ ] Exercise the generated vehicle module on desktop/mobile: image validation, required fields, recurrence, completion evidence, day-seven reminder, supervisor escalation, duplicate-cron protection, and department access.
- [ ] Upload/version/retire sample AdamsBrain documents and verify answers cite permitted sources, deny cross-department sources, reflect updated versions, and say when evidence is insufficient.
- [ ] Run `npm run build` as the final application gate and smoke-test existing sign-in, portal, safety calendar, LMS, Telegram, and Teams routes for regressions.

Risks/Considerations

- **Hermes is a high-privilege runtime.** Its API can execute tools on its host. It must be private, bearer-authenticated, sandboxed, denied unattended dangerous commands, and given no production database/API credentials. Only the build workspace receives a narrowly scoped GitHub credential.
- **Generated code remains supply-chain input.** Human approval alone is insufficient; enforce denied paths, typed contracts, deterministic validation, preview checks, immutable spec/SHA binding, and an audit trail before merge.
- **Vercel serverless requests are not build workers.** Generation/building must be asynchronous on the Hermes host; Vercel routes only enqueue, reconcile, approve, and display state.
- **The current authorization posture is prototype-grade.** Open Firestore settings/team-member rules, an open catch-all, client-side role assumptions, unguarded API routes, and credentials in history block safe production use until Step 1 and credential rotation are complete.
- **The current Ask page is mock data**, and existing AI routes are inconsistent (some bypass the shared LLM settings and call OpenAI directly). New Adams Harness code should use the Hermes adapter; broad legacy AI migration can follow without blocking the prototype.
- **“Runtime source code” is bounded in this prototype.** Hermes generates compiled module definitions/tests under an allowlist, while shared connectors/runtime stay human-reviewed. Allowing arbitrary generated React/server/security/dependency changes would materially expand risk and is intentionally out of scope.
- **Knowledge freshness is source-specific.** Uploads update when admins publish a new version; platform adapters can refresh on change/schedule. “Constantly updated” must not imply unsupervised ingestion of email or external content.
- **Public certification QR pages need a separate privacy review.** When implemented, expose only employee name/photo (if approved), active certification names, issuer, and validity—not internal IDs, training history, contact details, or documents.
- **Existing requested solutions are a roadmap, not all part of the first deploy.** The vehicle module proves the complete generation/deployment loop; the other three then reuse the same interview, module, AdamsBrain, notification, and channel infrastructure.